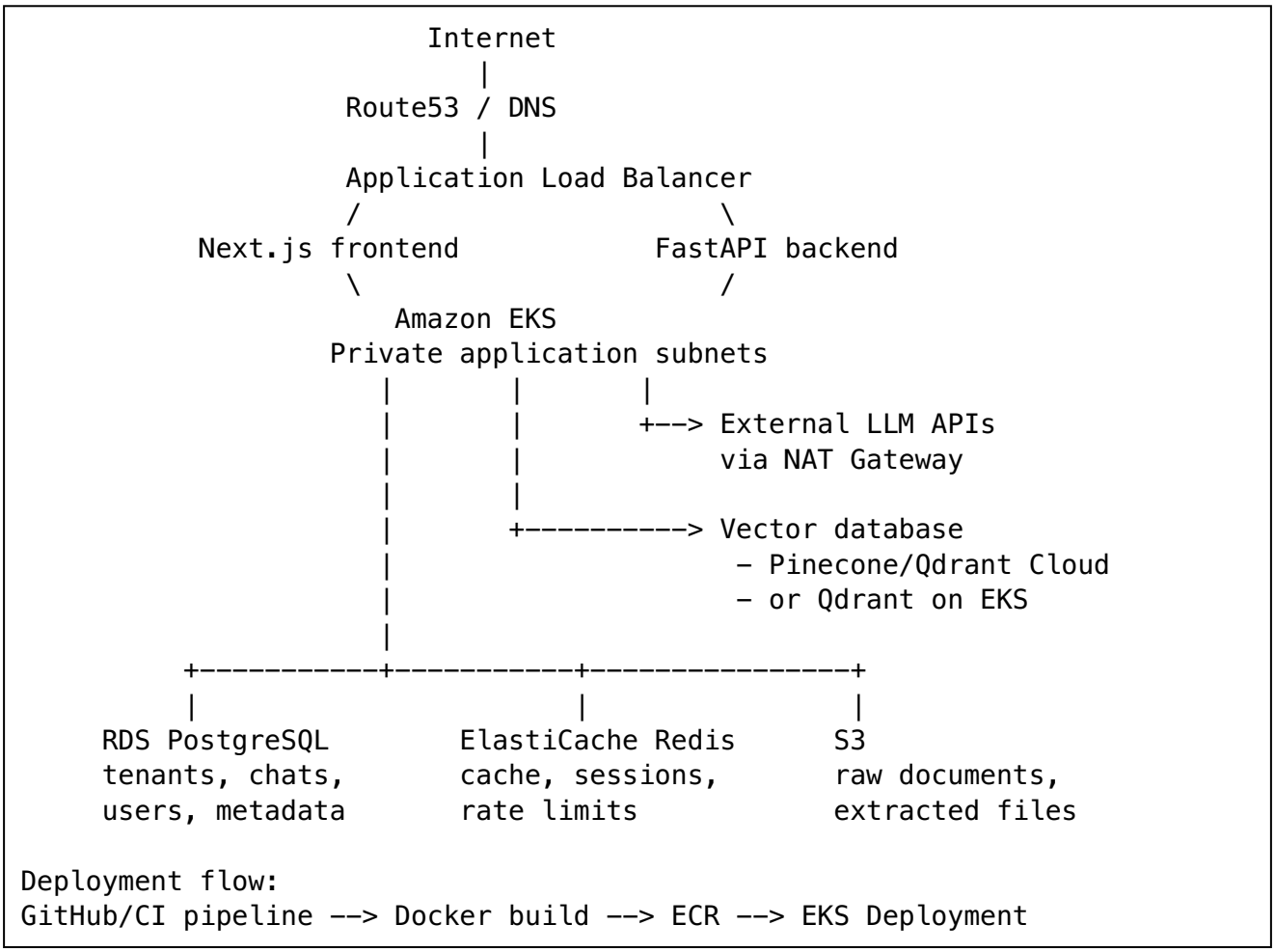
Day 19 – AWS GenAI Infrastructure with Terraform

1. Production GenAI SaaS architecture

A practical AWS architecture for a RAG or agentic SaaS could look like this:



The key security principle is:

> Only the load balancer should normally be directly internet-facing. EKS worker nodes, PostgreSQL, Redis, and self-hosted vector databases should remain in private subnets.

Amazon VPC provides the isolated network, EKS runs the containerized application, RDS stores transactional metadata, ElastiCache handles low-latency transient data, S3 stores documents, and ECR stores deployment images. (AWS Documentation)

---

2. Terraform provider foundation

These snippets are intentionally incomplete. In a real repository, they would be organized into modules such as `network`, `eks`, `database`, `cache`, `storage`, `iam`, and `container-registry`.

```
terraform {
  required_version = ">= 1.8.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
```

```
      version = "~> 6.0"
    }
  }

  backend "s3" {
    bucket       = "company-terraform-state"
    key          = "genai-saas/prod/terraform.tfstate"
    region       = "ap-south-1"
    encrypt      = true
    use_lockfile = true
  }
}

provider "aws" {
  region = var.aws_region

  default_tags {
    tags = {
      Project     = "genai-saas"
      Environment = var.environment
      ManagedBy   = "Terraform"
    }
  }
}
```

Pin the AWS provider to an accepted version range and test provider upgrades through CI rather than automatically accepting an untested major version. The HashiCorp AWS Provider manages resources such as VPC, EKS, RDS, S3, ECR, and ElastiCache. (GitHub)

---

3. VPC and networking

3.1 CIDR and subnet design

A CIDR defines the IP range available inside the VPC.

For example:

```
VPC: 10.20.0.0/16

Public subnets:
10.20.0.0/24   AZ-A
10.20.1.0/24   AZ-B

Private application subnets:
10.20.10.0/24  AZ-A
10.20.11.0/24  AZ-B

Private data subnets:
10.20.20.0/24  AZ-A
10.20.21.0/24  AZ-B
```

Each subnet exists in one Availability Zone. Using at least two Availability Zones prevents a single-AZ failure from taking down the complete application. (AWS Documentation)

Public subnets

Public subnets have a route to an Internet Gateway.

Typical resources:

- Internet-facing ALB
- NAT gateways
- Occasionally bastion hosts, although Systems Manager is generally preferable

Private application subnets

Typical resources:

- EKS worker nodes
- FastAPI pods
- Next.js pods
- Ingestion workers
- Self-hosted Qdrant pods

They can make outbound calls through a NAT gateway, but the internet cannot directly initiate connections to them.

Private data subnets

Typical resources:

- RDS PostgreSQL
- ElastiCache
- Self-hosted databases

These subnets should generally have no direct internet route.

A public NAT gateway is placed in a public subnet and allows resources in private subnets to initiate outbound connections without accepting unsolicited inbound internet traffic. (AWS Documentation)

3.2 Terraform VPC snippet

```
data "aws_availability_zones" "available" {
  state = "available"
}

locals {
  azs = slice(data.aws_availability_zones.available.names, 0, 2)
}

resource "aws_vpc" "main" {
  cidr_block           = "10.20.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "genai-${var.environment}"
  }
}
```

```
resource "aws_subnet" "public" {
  count = 2

  vpc_id                  = aws_vpc.main.id
  availability_zone       = local.azs[count.index]
  cidr_block              = cidrsubnet(aws_vpc.main.cidr_block, 8,
count.index)
  map_public_ip_on_launch = true

  tags = {
    Name                     = "public-${count.index + 1}"
    "kubernetes.io/role/elb" = "1"
  }
}

resource "aws_subnet" "private_app" {
  count = 2

  vpc_id            = aws_vpc.main.id
  availability_zone = local.azs[count.index]
  cidr_block        = cidrsubnet(aws_vpc.main.cidr_block, 8, count.index +
10)

  tags = {
    Name                              = "private-app-${count.index + 1}"
    "kubernetes.io/role/internal-elb" = "1"
  }
}

resource "aws_subnet" "private_data" {
  count = 2

  vpc_id            = aws_vpc.main.id
  availability_zone = local.azs[count.index]
  cidr_block        = cidrsubnet(aws_vpc.main.cidr_block, 8, count.index +
20)

  tags = {
    Name = "private-data-${count.index + 1}"
  }
}
```

The Kubernetes subnet tags help the AWS Load Balancer Controller distinguish subnets intended for internet-facing and internal load balancers.

3.3 Internet Gateway and NAT gateway

```
resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "genai-igw"
  }
```

```
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }
}

resource "aws_route_table_association" "public" {
  count = 2

  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}

resource "aws_eip" "nat" {
  count  = 2
  domain = "vpc"

  depends_on = [aws_internet_gateway.main]
}

resource "aws_nat_gateway" "main" {
  count = 2

  allocation_id = aws_eip.nat[count.index].id
  subnet_id     = aws_subnet.public[count.index].id

  depends_on = [aws_internet_gateway.main]
}

resource "aws_route_table" "private_app" {
  count = 2

  vpc_id = aws_vpc.main.id

  route {
    cidr_block     = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main[count.index].id
  }
}

resource "aws_route_table_association" "private_app" {
  count = 2

  subnet_id      = aws_subnet.private_app[count.index].id
  route_table_id = aws_route_table.private_app[count.index].id
}
```

For production, one NAT gateway per Availability Zone avoids cross-AZ dependency. For development, teams sometimes use one NAT gateway to reduce cost, accepting lower availability.

Private data subnets would use route tables without a default internet route.

3.4 Reduce NAT usage with VPC endpoints

EKS workloads frequently access S3 and ECR. Without VPC endpoints, some of this traffic may pass through NAT gateways.

An S3 gateway endpoint allows private connectivity to S3:

```
resource "aws_vpc_endpoint" "s3" {
  vpc_id            = aws_vpc.main.id
  service_name      = "com.amazonaws.${var.aws_region}.s3"
  vpc_endpoint_type = "Gateway"

  route_table_ids = [
    for route_table in aws_route_table.private_app :
    route_table.id
  ]
}
```

In a mature implementation, consider endpoints for:

- S3
- ECR API
- ECR Docker registry
- CloudWatch Logs
- Secrets Manager
- Systems Manager
- STS

This can reduce NAT traffic and allow tighter private networking.

---

4. Security groups versus NACLs

Feature	Security group	Network ACL
Applied to	ENI/resource	Entire subnet
Behavior	Stateful	Stateless
Rules	Allow rules	Allow and deny rules
Return traffic	Automatically allowed	Must be explicitly allowed
Typical use	Primary application firewall	Broad subnet-level guardrail

Security groups should normally be the main control for application traffic. NACLs are useful for coarse-grained subnet controls or explicit deny requirements. (AWS Documentation)

Example traffic rules

```
Internet
   |
   | TCP 443
   v
```

```
ALB security group
   |
   | TCP 3000 and 8000
   v
EKS application security group
   |
   +---- TCP 5432 ----> RDS security group
   |
   +---- TCP 6379 ----> Redis security group
   |
   +---- TCP 443 -----> S3, ECR, LLM APIs, vector DB APIs
```

Terraform:

```
resource "aws_security_group" "eks_apps" {
  name   = "genai-eks-apps"
  vpc_id = aws_vpc.main.id

  egress {
    description = "Application outbound access"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "rds" {
  name   = "genai-rds"
  vpc_id = aws_vpc.main.id
}

resource "aws_vpc_security_group_ingress_rule" "rds_from_eks" {
  security_group_id            = aws_security_group.rds.id
  referenced_security_group_id = aws_security_group.eks_apps.id

  ip_protocol = "tcp"
  from_port   = 5432
  to_port     = 5432
}

resource "aws_security_group" "redis" {
  name   = "genai-redis"
  vpc_id = aws_vpc.main.id
}

resource "aws_vpc_security_group_ingress_rule" "redis_from_eks" {
  security_group_id            = aws_security_group.redis.id
  referenced_security_group_id = aws_security_group.eks_apps.id

  ip_protocol = "tcp"
  from_port   = 6379
```

```
  to_port     = 6379
}
```

Notice that PostgreSQL and Redis do not allow a CIDR such as `0.0.0.0/0`. They allow connections only from the EKS application security group.

---

5. Amazon EKS for GenAI services

5.1 Control plane versus worker nodes

EKS control plane

AWS manages:

- Kubernetes API server
- `etcd`
- Scheduler
- Controller manager
- Control-plane availability and patching

Worker nodes

Worker nodes run:

- FastAPI pods
- Next.js pods
- RAG ingestion workers
- Background agent workers
- Embedding jobs
- Optional self-hosted model or Qdrant pods

EKS managed node groups automate provisioning and lifecycle operations for EC2 worker nodes. (AWS Documentation)

5.2 Separate node groups by workload

A useful production pattern is:

```
system node group
  CoreDNS, controllers, monitoring agents

application node group
  FastAPI, Next.js, lightweight workers

memory-optimized node group
  rerankers, large document-processing jobs

GPU node group
  self-hosted embedding or inference models

Spot node group
  interruptible asynchronous ingestion jobs
```

Use Kubernetes labels, taints, tolerations, and node selectors so expensive GPU nodes are not used by ordinary API pods.

5.3 EKS IAM roles

The cluster role allows EKS to manage AWS resources. The worker-node role allows EC2 nodes to connect to the cluster and pull images.

```
data "aws_iam_policy_document" "eks_cluster_assume" {
  statement {
    actions = ["sts:AssumeRole"]

    principals {
      type        = "Service"
      identifiers = ["eks.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "eks_cluster" {
  name               = "genai-eks-cluster-role"
  assume_role_policy = data.aws_iam_policy_document.eks_cluster_assume.json
}

resource "aws_iam_role_policy_attachment" "eks_cluster" {
  role       = aws_iam_role.eks_cluster.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
}

data "aws_iam_policy_document" "eks_node_assume" {
  statement {
    actions = ["sts:AssumeRole"]

    principals {
      type        = "Service"
      identifiers = ["ec2.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "eks_node" {
  name               = "genai-eks-node-role"
  assume_role_policy = data.aws_iam_policy_document.eks_node_assume.json
}

resource "aws_iam_role_policy_attachment" "eks_worker" {
  role       = aws_iam_role.eks_node.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy"
}

resource "aws_iam_role_policy_attachment" "ecr_pull" {
  role       = aws_iam_role.eks_node.name
```

```
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryPullOnly"
}
```

`AmazonEKSClusterPolicy` is designed for the cluster role, `AmazonEKSWorkerNodePolicy` allows nodes to connect to EKS, and `AmazonEC2ContainerRegistryPullOnly` grants image-pull permissions. (AWS Documentation)

5.4 EKS cluster and node group

```
resource "aws_eks_cluster" "main" {
  name     = "genai-${var.environment}"
  role_arn = aws_iam_role.eks_cluster.arn
  version  = var.eks_version

  enabled_cluster_log_types = [
    "api",
    "audit",
    "authenticator",
    "controllerManager",
    "scheduler"
  ]

  vpc_config {
    subnet_ids = aws_subnet.private_app[*].id

    endpoint_private_access = true
    endpoint_public_access  = true
    public_access_cidrs     = var.admin_cidrs
  }

  depends_on = [
    aws_iam_role_policy_attachment.eks_cluster
  ]
}
```

For production, the public Kubernetes API endpoint should either be disabled or restricted to known office, VPN, or CI/CD network ranges.

A managed node group:

```
resource "aws_launch_template" "eks_apps" {
  name_prefix = "genai-apps-"

  vpc_security_group_ids = [
    aws_eks_cluster.main.vpc_config[0].cluster_security_group_id,
    aws_security_group.eks_apps.id
  ]

  metadata_options {
    http_endpoint               = "enabled"
    http_tokens                 = "required"
    http_put_response_hop_limit = 1
  }
```

```
}

resource "aws_eks_node_group" "apps" {
  cluster_name    = aws_eks_cluster.main.name
  node_group_name = "application"
  node_role_arn   = aws_iam_role.eks_node.arn
  subnet_ids      = aws_subnet.private_app[*].id

  instance_types = ["m7i.large"]
  capacity_type  = "ON_DEMAND"

  scaling_config {
    min_size     = 2
    desired_size = 2
    max_size     = 10
  }

  update_config {
    max_unavailable = 1
  }

  labels = {
    workload = "application"
  }

  launch_template {
    id      = aws_launch_template.eks_apps.id
    version = aws_launch_template.eks_apps.latest_version
  }

  depends_on = [
    aws_iam_role_policy_attachment.eks_worker,
    aws_iam_role_policy_attachment.ecr_pull
  ]
}
```

The EC2 instance type here is illustrative. Capacity should be selected from measured CPU, memory, network, concurrency, and workload characteristics rather than copied from a template.

5.5 Autoscaling

Three separate scaling layers may exist:

Horizontal Pod Autoscaler

Scales the number of FastAPI or worker pods based on metrics such as:

- CPU
- Memory
- Request concurrency
- Queue depth
- LLM request backlog
- Token-processing rate

Node autoscaling

Cluster Autoscaler, Karpenter, or an AWS-managed mechanism can add nodes when pods cannot be scheduled.

Application-level concurrency

FastAPI worker count, asynchronous task queues, LLM provider rate limits, database connection pools, and Redis limits must also be controlled.

A common mistake is scaling pods from 5 to 100 while leaving the database connection pool unconstrained. That can overwhelm PostgreSQL even when EKS itself is healthy.

---

6. EKS and ALB Ingress

The AWS Load Balancer Controller watches Kubernetes Ingress and Service resources and provisions AWS load balancers. ALB handles Layer 7 HTTP/HTTPS routing and can route different paths to different Kubernetes services. (AWS Documentation)

For example:

```
https://app.example.com/       -> nextjs-web:3000
https://app.example.com/api/*  -> rag-api:8000
```

Illustrative Terraform Kubernetes resource:

```
resource "kubernetes_ingress_v1" "public" {
  metadata {
    name      = "genai-public"
    namespace = "genai"

    annotations = {
      "alb.ingress.kubernetes.io/scheme"      = "internet-facing"
      "alb.ingress.kubernetes.io/target-type" = "ip"
      "alb.ingress.kubernetes.io/listen-ports" = jsonencode([
        { HTTPS = 443 }
      ])
      "alb.ingress.kubernetes.io/certificate-arn" = var.acm_certificate_arn
    }
  }

  spec {
    ingress_class_name = "alb"

    rule {
      host = "app.example.com"

      http {
        path {
          path      = "/api"
          path_type = "Prefix"

          backend {
            service {
              name = "rag-api"
```

```
              port {
                number = 8000
              }
            }
          }
        }

        path {
          path      = "/"
          path_type = "Prefix"

          backend {
            service {
              name = "nextjs-web"

              port {
                number = 3000
              }
            }
          }
        }
      }
    }
  }
}
```

The controller itself needs an IAM role, preferably through EKS Pod Identity or IRSA rather than permissions attached broadly to every worker node. AWS recommends installing the controller rather than relying on the legacy Kubernetes AWS load-balancer behavior. (AWS Documentation)

---

7. RDS PostgreSQL

7.1 What PostgreSQL stores in a GenAI SaaS

RDS PostgreSQL is suitable for transactional and relational information such as:

- Users and organizations
- Tenant configuration
- RBAC and permissions
- Conversation and message history
- Document metadata
- Ingestion job status
- Prompt and model configurations
- Agent execution metadata
- Billing and usage records
- Evaluation results
- References to S3 object keys
- References to vector IDs

A vector database normally stores embeddings and searchable payloads. PostgreSQL stores the business truth around those vectors.

Example:

```
PostgreSQL document row:
document_id: doc-123
tenant_id: tenant-55
s3_key: tenants/tenant-55/documents/doc-123.pdf
vector_collection: tenant-55-documents
vector_ids: chunk-1 ... chunk-80
status: indexed
```

RDS for PostgreSQL supports VPC deployment, backups, snapshots, Multi-AZ configurations, read replicas, and SSL connectivity. (AWS Documentation)

7.2 RDS subnet group

```
resource "aws_db_subnet_group" "postgres" {
  name       = "genai-postgres"
  subnet_ids = aws_subnet.private_data[*].id

  tags = {
    Name = "genai-postgres"
  }
}
```

The subnet group tells RDS which private data subnets it may use.

7.3 RDS Terraform snippet

```
resource "aws_db_instance" "postgres" {
  identifier = "genai-${var.environment}"

  engine         = "postgres"
  instance_class = "db.r7g.large"

  db_name  = "genai"
  username = "app_admin"

  manage_master_user_password = true

  allocated_storage     = 100
  max_allocated_storage = 500
  storage_type          = "gp3"
  storage_encrypted     = true

  db_subnet_group_name   = aws_db_subnet_group.postgres.name
  vpc_security_group_ids = [aws_security_group.rds.id]

  publicly_accessible = false
  multi_az            = true
```

```
  backup_retention_period = 7
  deletion_protection      = true
  skip_final_snapshot      = false

  performance_insights_enabled = true
}
```

Important production settings:

- `publicly_accessible = false`
- Encryption enabled
- Backups enabled
- Deletion protection
- Multi-AZ for important environments
- Credentials managed through Secrets Manager
- Restricted security-group source
- Monitoring and slow-query analysis

RDS encryption covers database storage, logs, backups, read replicas, and snapshots. Multi-AZ deployments provide failover support; a traditional single-standby Multi-AZ DB instance does not use its standby to serve read traffic. (AWS Documentation)

Application configuration

The FastAPI deployment should receive the database endpoint through configuration:

```
DATABASE_HOST = <RDS endpoint>
DATABASE_PORT = 5432
DATABASE_NAME = genai
DATABASE_SECRET_ARN = <Secrets Manager ARN>
```

Do not hard-code the password in a Docker image, Git repository, Terraform variable file, or Kubernetes Deployment.

---

8. ElastiCache for Redis

8.1 GenAI use cases

Redis is useful for:

Response caching

```
Key:
llm-response:{tenant}:{model}:{prompt_hash}

Value:
Generated answer and metadata
```

Semantic caching

Embed the incoming question and reuse a previous answer when semantic similarity is above a controlled threshold.

AWS documents semantic caching as a GenAI use case that can reduce LLM cost and latency. (AWS Documentation)

Rate limiting

```
rate-limit:{tenant}:{minute}
rate-limit:{user}:{model}
```

Session and temporary state

- OAuth session state
- Agent intermediate state
- WebSocket connection metadata
- Temporary tool results
- Distributed locks

Queue or coordination state

Redis can coordinate short-lived background jobs, but durable business state should remain in PostgreSQL or another durable system.

8.2 Redis subnet group

```
resource "aws_elasticache_subnet_group" "redis" {
  name       = "genai-redis"
  subnet_ids = aws_subnet.private_data[*].id
}
```

8.3 Redis replication group

```
variable "redis_auth_token" {
  type      = string
  sensitive = true
}

resource "aws_elasticache_replication_group" "redis" {
  replication_group_id = "genai-${var.environment}"
  description          = "Redis for GenAI API caching and rate limiting"

  engine    = "redis"
  node_type = "cache.r7g.large"
  port      = 6379

  num_cache_clusters         = 2
  automatic_failover_enabled = true
  multi_az_enabled           = true

  transit_encryption_enabled = true
  at_rest_encryption_enabled = true
  auth_token                 = var.redis_auth_token

  subnet_group_name  = aws_elasticache_subnet_group.redis.name
  security_group_ids = [aws_security_group.redis.id]
```

```
  snapshot_retention_limit = 7
}
```

ElastiCache is a managed in-memory service compatible with Redis OSS and Valkey. It removes much of the operational work involved in deploying and scaling an in-memory cache. (AWS Documentation)

Important Terraform concern

Although a variable is marked `sensitive`, its value can still exist in Terraform state. Therefore:

- Encrypt remote state
- Restrict state-bucket access
- Avoid exposing state in CI logs
- Rotate credentials
- Prefer supported managed-secret integrations where available

---

9. S3 for RAG documents

9.1 What belongs in S3

S3 is appropriate for:

- Uploaded PDF, DOCX, HTML, image, and text files
- Extracted document text
- OCR output
- Chunking artifacts
- Evaluation datasets
- Prompt templates
- Generated exports
- Model or adapter artifacts
- Ingestion error payloads

Example key structure:

```
s3://genai-documents/
  tenants/
    tenant-123/
      documents/
        document-456/
          original.pdf
          extracted.json
          chunks.jsonl
```

The database stores the object key and document metadata. It should not normally store the entire binary document.

9.2 Terraform S3 configuration

```
data "aws_caller_identity" "current" {}

resource "aws_s3_bucket" "documents" {
```

```
  bucket = "genai-${data.aws_caller_identity.current.account_id}-$
{var.environment}-documents"
}

resource "aws_s3_bucket_versioning" "documents" {
  bucket = aws_s3_bucket.documents.id

  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "documents" {
  bucket = aws_s3_bucket.documents.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "documents" {
  bucket = aws_s3_bucket.documents.id

  block_public_acls       = true
  ignore_public_acls      = true
  block_public_policy     = true
  restrict_public_buckets = true
}
```

S3 encrypts new uploads by default with SSE-S3, but explicitly defining encryption in Terraform documents and enforces the intended configuration. S3 Block Public Access should remain enabled for private RAG documents. (AWS Documentation)

Upload pattern

A common secure flow is:

```
1. Frontend requests upload permission from FastAPI.
2. FastAPI checks tenant and user authorization.
3. FastAPI generates a short-lived presigned S3 URL.
4. Browser uploads directly to S3.
5. An ingestion job processes the S3 object.
6. Extracted chunks are embedded and written to the vector DB.
7. PostgreSQL document status changes to INDEXED.
```

This prevents the FastAPI application from becoming a bottleneck for large file uploads.

---

10. ECR for backend and frontend images

Use separate repositories:

```
genai-api
genai-web
genai-ingestion-worker
```

Terraform:

```
locals {
  repositories = toset([
    "genai-api",
    "genai-web",
    "genai-ingestion-worker"
  ])
}

resource "aws_ecr_repository" "services" {
  for_each = local.repositories

  name                 = each.value
  image_tag_mutability = "IMMUTABLE"

  image_scanning_configuration {
    scan_on_push = true
  }

  encryption_configuration {
    encryption_type = "AES256"
  }
}
```

A lifecycle policy prevents unlimited accumulation of old images:

```
resource "aws_ecr_lifecycle_policy" "services" {
  for_each = aws_ecr_repository.services

  repository = each.value.name

  policy = jsonencode({
    rules = [
      {
        rulePriority = 1
        description  = "Remove excess untagged images"

        selection = {
          tagStatus   = "untagged"
          countType   = "imageCountMoreThan"
          countNumber = 30
        }

        action = {
          type = "expire"
        }
      }
    ]
```

```
  })
}
```

ECR scanning identifies vulnerabilities in container images, while lifecycle policies remove or archive unused images based on configured criteria. (AWS Documentation)

Deployment flow

```
Developer pushes code
        |
        v
CI pipeline runs tests and scans
        |
        v
Docker image built
        |
        v
Image pushed with immutable tag:
genai-api:git-a84f21c
        |
        v
Kubernetes Deployment updated
        |
        v
EKS nodes pull image from ECR
        |
        v
Readiness probe succeeds
        |
        v
ALB sends traffic to new pods
```

Avoid deploying mutable tags such as only `latest`. Prefer a Git SHA, build number, or immutable release version.

---

11. IAM: cluster, nodes, and pods

A strong interview answer distinguishes three IAM layers.

11.1 Cluster IAM role

Used by the EKS control plane to manage required AWS resources.

11.2 Node IAM role

Used by EC2 worker nodes for operations such as:

- Joining the EKS cluster
- Pulling images from ECR
- Node-level AWS integration

It should not automatically have access to every application S3 bucket or secret.

11.3 Pod or workload IAM role

Used by a particular Kubernetes service account.

Examples:

```
rag-api service account
  S3 read/write for tenant documents
  Secrets Manager read for application secrets

ingestion-worker service account
  S3 read
  S3 write to extracted-artifact prefix
  SQS consume permissions

cloudwatch-agent service account
  CloudWatch logs and metrics permissions

load-balancer-controller service account
  ALB-related EC2 and ELB permissions
```

EKS Pod Identity associates an IAM role with a Kubernetes service account so that application pods receive temporary AWS credentials without storing long-lived access keys. IRSA remains another common service-account identity mechanism. (AWS Documentation)

11.4 Pod Identity Terraform example

First create an IAM role that trusts the EKS Pod Identity service:

```
data "aws_iam_policy_document" "rag_api_assume" {
  statement {
    actions = [
      "sts:AssumeRole",
      "sts:TagSession"
    ]

    principals {
      type        = "Service"
      identifiers = ["pods.eks.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "rag_api" {
  name               = "genai-rag-api"
  assume_role_policy = data.aws_iam_policy_document.rag_api_assume.json
}
```

The EKS Pod Identity trust relationship uses the `pods.eks.amazonaws.com` service principal and requires both `sts:AssumeRole` and `sts:TagSession`. (AWS Documentation)

Create a least-privilege S3 policy:

```
data "aws_iam_policy_document" "rag_api_s3" {
  statement {
    sid       = "ListDocumentBucket"
    actions   = ["s3:ListBucket"]
    resources = [aws_s3_bucket.documents.arn]
  }

  statement {
    sid = "ReadWriteDocumentObjects"

    actions = [
      "s3:GetObject",
      "s3:PutObject"
    ]

    resources = [
      "${aws_s3_bucket.documents.arn}/tenants/*"
    ]
  }
}

resource "aws_iam_policy" "rag_api_s3" {
  name   = "genai-rag-api-s3"
  policy = data.aws_iam_policy_document.rag_api_s3.json
}

resource "aws_iam_role_policy_attachment" "rag_api_s3" {
  role       = aws_iam_role.rag_api.name
  policy_arn = aws_iam_policy.rag_api_s3.arn
}
```

Install the Pod Identity agent and create the association:

```
resource "aws_eks_addon" "pod_identity_agent" {
  cluster_name = aws_eks_cluster.main.name
  addon_name   = "eks-pod-identity-agent"
}

resource "aws_eks_pod_identity_association" "rag_api" {
  cluster_name    = aws_eks_cluster.main.name
  namespace       = "genai"
  service_account = "rag-api"
  role_arn        = aws_iam_role.rag_api.arn

  depends_on = [
    aws_eks_addon.pod_identity_agent
  ]
}
```

The Kubernetes Deployment then uses:

```
spec:
  serviceAccountName: rag-api
```

The AWS SDK inside the FastAPI container uses its normal default credential chain. No hard-coded access key is needed.

---

12. React and Next.js deployment choices

Option A: Next.js on EKS

Use this when the application needs:

- Server-side rendering
- Next.js API routes
- Dynamic middleware
- Server-side authentication
- Runtime page generation

Flow:

```
Browser -> ALB -> Next.js pod -> FastAPI pod
```

Both frontend and backend images are stored in ECR.

Option B: Static React or exported Next.js

Use:

```
Browser -> CloudFront -> S3 static assets
                         |
                         +--> ALB -> FastAPI
```

This is often cheaper and simpler when server-side rendering is unnecessary.

For today's EKS-focused architecture, assume that the Next.js application is containerized and runs in EKS.

---

13. Vector database integration

13.1 Pinecone or managed Qdrant

A managed vector database lives outside the AWS resources managed in this design.

```
FastAPI pod
   |
   | HTTPS 443 through NAT
   v
Managed vector DB endpoint
```

Store:

- Endpoint in application configuration
- API key in Secrets Manager
- Collection or namespace mapping in PostgreSQL

Tenant isolation approaches include:

```
One namespace per tenant
One collection per large tenant
Shared collection with mandatory tenant_id filtering
```

Never rely only on the prompt to enforce tenant isolation. Enforce tenant filters in application code and retrieval queries.

13.2 Self-hosted Qdrant on EKS

```
FastAPI -> private Qdrant Kubernetes Service
                    |
                Qdrant pods
                  |
             Persistent EBS volumes
```

For production:

- Use a dedicated node group
- Use persistent volumes
- Apply pod anti-affinity
- Configure backups
- Use a private `ClusterIP` service
- Do not expose Qdrant directly to the internet
- Monitor disk usage and indexing latency

Self-hosting gives infrastructure control but transfers availability, upgrades, backups, scaling, and recovery responsibility to your team.

---

14. End-to-end request flow

Consider a user asking:

> "Summarize the termination clause in my contract."

Step 1: Frontend request

The browser sends:

```
POST /api/chat
```

Route53 resolves the domain to the ALB.

Step 2: ALB routing

The ALB terminates TLS and forwards `/api` to the FastAPI Kubernetes Service.

Step 3: Authentication

FastAPI validates the user token and determines:

```
user_id
tenant_id
roles
subscription
```

Step 4: Rate limiting

FastAPI checks Redis:

```
rate-limit:{tenant_id}:{minute}
```

If the tenant exceeds its allowed request rate, the API returns HTTP 429.

Step 5: Conversation state

FastAPI reads from RDS:

```
conversation
previous messages
tenant configuration
allowed models
document permissions
```

Step 6: Retrieval

FastAPI sends the question embedding and mandatory tenant filter to Pinecone or Qdrant.

```
query_vector = embed(question)

filter:
tenant_id = current_user.tenant_id
document_id in authorized_documents
```

Step 7: Document lookup

Vector results contain:

```
chunk_id
document_id
text
score
page_number
s3_object_key
```

The raw source document remains in S3.

Step 8: LLM invocation

FastAPI constructs the prompt and calls:

- An external LLM API through the NAT gateway, or
- A self-hosted inference service inside EKS

Step 9: Cache

Where safe, the answer or intermediate result is cached in Redis using a key that includes:

```
tenant
model
prompt version
retrieval configuration
document version
```

Step 10: Persistence

FastAPI stores in PostgreSQL:

```
user question
assistant answer
citations
model
token usage
latency
retrieval IDs
prompt version
```

Step 11: Response

FastAPI streams the answer through the ALB to the frontend.

Step 12: Observability

Application and platform signals are sent to CloudWatch or an external observability platform:

```
Request latency
LLM latency
Retrieval latency
Token usage
Cache hit rate
Error rate
Rate-limit events
Database connections
Pod restarts
Node pressure
```

---

15. Resource wiring in Terraform

Terraform creates the dependency graph from references:

```
aws_vpc.main
    |
    +--> aws_subnet.public
    +--> aws_subnet.private_app
    +--> aws_subnet.private_data
              |
              +--> aws_db_subnet_group.postgres
              |          |
              |          +--> aws_db_instance.postgres
              |
              +--> aws_elasticache_subnet_group.redis
                         |
                         +--> aws_elasticache_replication_group.redis

aws_subnet.private_app
    |
    +--> aws_eks_cluster.main
              |
```

```
              +--> aws_eks_node_group.apps
              +--> aws_eks_pod_identity_association.rag_api

aws_s3_bucket.documents
    |
    +--> aws_iam_policy.rag_api_s3
              |
              +--> aws_iam_role.rag_api
```

For example:

```
db_subnet_group_name = aws_db_subnet_group.postgres.name
```

creates an implicit dependency from the database instance to its subnet group.

Use `depends_on` only for dependencies Terraform cannot infer through references.

---

16. Production security checklist

Network

- ALB in public subnets
- EKS nodes in private application subnets
- RDS and Redis in private data subnets
- No public RDS endpoint
- No public Redis endpoint
- Restrict EKS API endpoint
- Use TLS for external and internal sensitive traffic
- Prefer VPC endpoints for AWS services
- Restrict outbound access where practical

IAM

- No static AWS credentials in containers
- Separate node and workload roles
- Pod Identity or IRSA for applications
- Separate IAM roles for FastAPI, workers, controllers, and observability
- Limit S3 access to required buckets and prefixes
- CI role can push images; node role normally needs only pull access

Data

- S3 Block Public Access
- RDS encryption
- Redis encryption in transit and at rest
- ECR image scanning
- Immutable image tags
- Secrets Manager for database, vector DB, and LLM credentials
- Tenant filtering at database and vector-query layers

Reliability

- Multiple Availability Zones
- At least two application nodes
- Multi-AZ RDS for production
- Redis replication and automatic failover
- RDS backups
- S3 versioning
- Qdrant or vector DB backup strategy
- Kubernetes readiness and liveness probes
- Pod disruption budgets
- Controlled rollout and rollback

---

17. Common mistakes

1. Placing worker nodes in public subnets

Worker nodes usually do not need public IP addresses. Put them in private subnets and expose applications through an ALB.

2. Making RDS publicly accessible

The FastAPI application should connect using private VPC networking.

3. Giving the node role full S3 access

Every pod on the node might gain unintended access. Use workload-specific Pod Identity or IRSA.

4. Using Redis as the permanent system of record

Redis is ideal for transient, low-latency data. Store durable tenant and conversation records in PostgreSQL.

5. Treating S3 as a vector database

S3 stores source objects. Pinecone, Qdrant, pgvector, or another vector engine performs similarity search.

6. Using only `latest` Docker tags

Immutable Git SHA or release tags make rollback and incident investigation much easier.

7. Scaling API pods without protecting PostgreSQL

Use controlled connection pools and consider a database proxy when connection counts become difficult to manage.

8. Logging sensitive prompts blindly

Prompts may contain PII, credentials, proprietary documents, or regulated information. Redact and control retention.

9. Sharing one environment

Development, staging, and production should have separate state and usually separate databases, caches, buckets, and clusters—or at minimum strong account and network separation.

---

18. Interview Q&A

1. Why place EKS worker nodes in private subnets?

Worker nodes do not need direct inbound internet access. An ALB exposes selected services, while nodes use NAT or VPC endpoints for outbound access.

2. What is the difference between the EKS control plane and worker nodes?

AWS manages the Kubernetes control plane. Worker nodes are the EC2 or managed compute instances where application pods actually execute.

3. Why use RDS PostgreSQL in a RAG platform?

RDS stores relational and transactional data such as users, tenants, conversations, document metadata, ingestion state, permissions, and usage records.

4. Why not store all data in the vector database?

Vector databases optimize similarity search, not general transactions, joins, billing records, permissions, or relational business constraints. PostgreSQL remains the system of record.

5. Where would Redis help in an LLM application?

Redis supports rate limiting, session storage, response caching, semantic caching, distributed locks, and temporary agent state.

6. How does an EKS application securely access S3?

Assign a least-privilege IAM role to the application's Kubernetes service account through EKS Pod Identity or IRSA. Do not store static AWS keys inside the pod.

7. How does ALB integrate with EKS?

The AWS Load Balancer Controller watches Kubernetes Ingress resources and provisions an ALB with listeners, target groups, routing rules, and pod or node targets.

8. What is the difference between a security group and a NACL?

A security group is stateful and applies to network interfaces or resources. A NACL is stateless and applies to a complete subnet.

9. How would you deploy a new FastAPI version?

Build and test a Docker image, scan it, push it to ECR with an immutable tag, update the Kubernetes Deployment, wait for readiness checks, and use a rolling, canary, or blue-green rollout.

10. How would you separate development, staging, and production?

Use separate Terraform state files and preferably separate AWS accounts or VPCs. Each environment should have independent EKS, RDS, Redis, S3, IAM, and configuration boundaries.

Interview-ready summary

> I would deploy the public entry point through an ALB, while running FastAPI, Next.js, and ingestion workers on EKS nodes in private application subnets. RDS PostgreSQL and ElastiCache would run in private data subnets and accept traffic only from approved EKS security groups. S3 would store raw RAG documents, ECR would store immutable container images, and EKS Pod Identity would provide workload-specific AWS permissions. Managed vector databases and external LLM APIs would be reached over controlled HTTPS egress, while Terraform would consistently provision and wire the complete environment.